

# MERN STACK INTERNSHIP – DAY 11 ASSIGNMENT REPORT

## Company

Insta Dot Analytics

## Submitted By

Name: Chayan Soni

Roll No: PU02325EUGBTCS086

Course: B.Tech (CSIT), First Year

College: Symbiosis University of Applied Sciences

Submission Date: 12-06-2026

## FOR SOURCE CODE:

**GIT HUB** :- <https://github.com/chayan-soni/MERN-Internship>

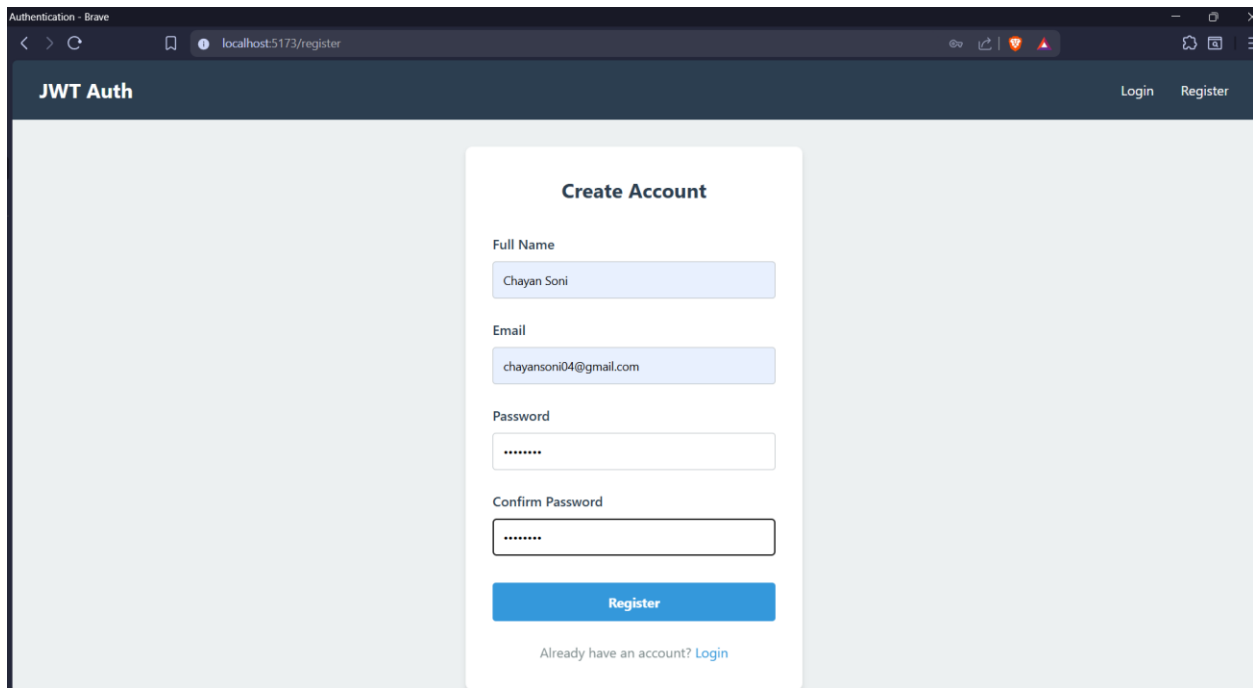
## Screenshots

### Screenshot 1: User Registration Page

#### **Description:**

This screen allows new users to create an account by entering their name, email address, and password. Input validation is implemented to ensure proper data entry before registration.

## Screenshot:



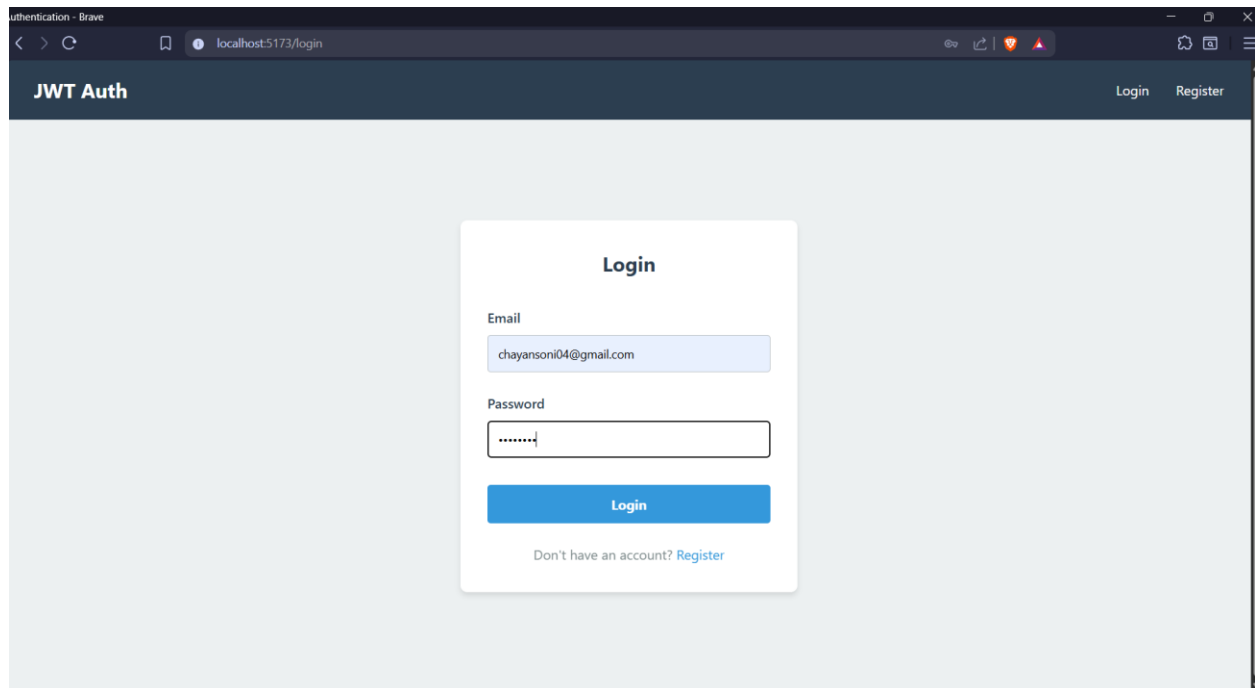
The screenshot shows a web browser window titled "Authentication - Brave" with the address bar displaying "localhost:5173/register". The page has a dark blue header with the text "JWT Auth" on the left and "Login Register" on the right. The main content area is light gray and features a white "Create Account" form in the center. The form includes the following fields: "Full Name" (containing "Chayan Soni"), "Email" (containing "chayansoni04@gmail.com"), "Password" (masked with "\*\*\*\*\*"), and "Confirm Password" (masked with "\*\*\*\*\*"). Below these fields is a blue "Register" button. At the bottom of the form, there is a link that says "Already have an account? [Login](#)".

## Screenshot 2: User Login Page

### Description:

This screen enables registered users to authenticate themselves using their email and password. Upon successful login, a JWT token is generated and stored for session management.

## Screenshot:

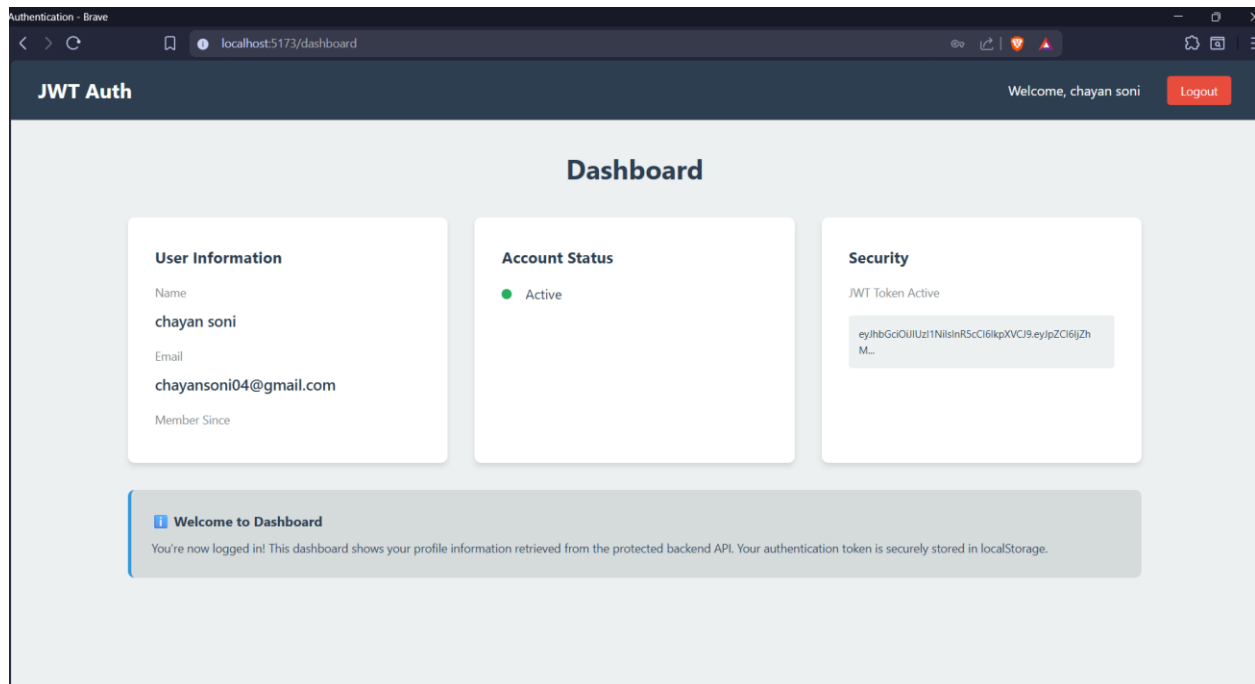


## Screenshot 3: Dashboard Page

### Description:

This is the protected dashboard page that can only be accessed by authenticated users. It confirms successful login and demonstrates route protection functionality.

## Screenshot:

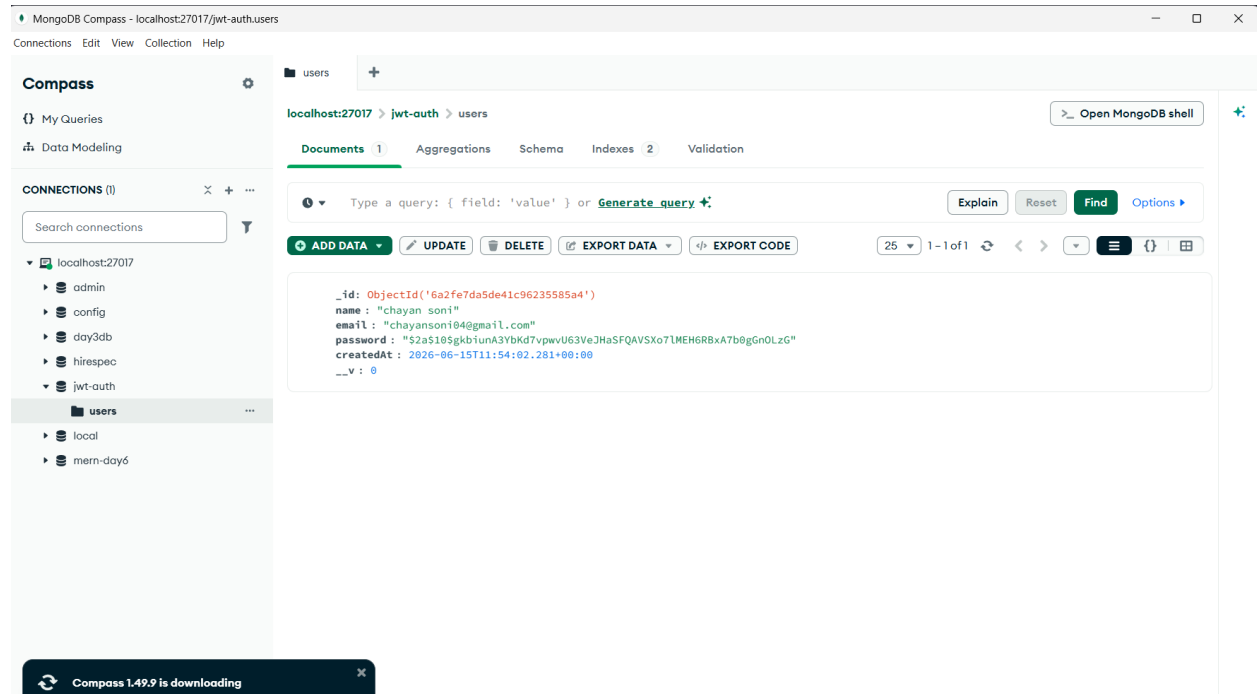


## Screenshot 4: MongoDB Database

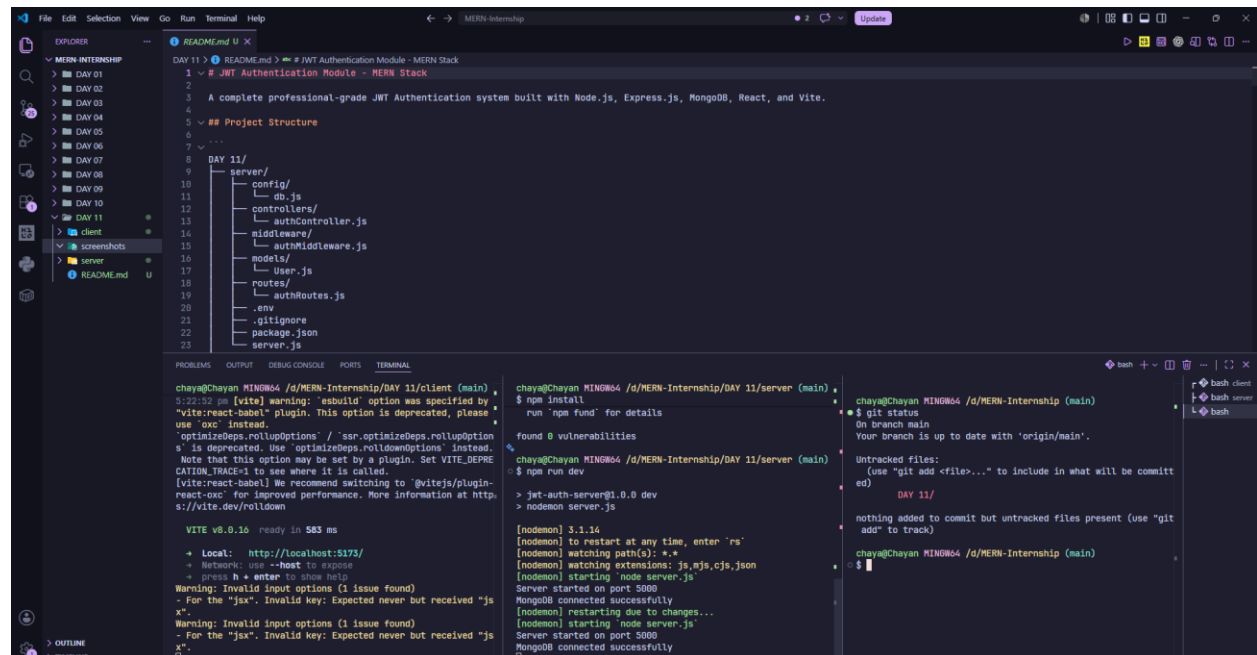
### Description:

This screenshot displays the user records stored in the MongoDB database. Passwords are securely stored in hashed format using bcryptjs.

## Screenshot:



## Screenshot 5: Protected Route Verification



## Objective

The objective of this assignment was to build a secure User Authentication Module using the MERN stack. The application allows users to register, log in, and access protected resources using JSON Web Tokens (JWT). The project focuses on implementing authentication, authorization, route protection, and secure password management.

## Technologies Used

### Frontend

- React.js (Vite)
- React Router DOM
- Axios
- Context API
- CSS

### Backend

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT (JSON Web Token)
- bcryptjs
- dotenv
- cors

### Tools

- Git
- GitHub
- Visual Studio Code

## Features Implemented

### User Registration

- Created a registration form for new users.
- Implemented input validation.

- Stored user details securely in MongoDB.
- Hashed passwords using bcryptjs before storing them in the database.

### User Login

- Developed a login system using email and password.
- Verified user credentials.
- Generated JWT tokens upon successful authentication.

### Authentication & Authorization

- Implemented JWT-based authentication.
- Protected backend routes using authentication middleware.
- Allowed access only to authenticated users.

### Protected Routes

- Created protected routes in React.
- Redirected unauthenticated users to the login page.
- Maintained user sessions using local storage.

### Dashboard

- Developed a dashboard page accessible only after successful login.
- Displayed authenticated user information.
- Added logout functionality.

### Project Workflow

1. User registers through the registration form.
2. Password is encrypted using bcryptjs.
3. User data is stored in MongoDB.
4. User logs in with valid credentials.
5. Server verifies credentials and generates a JWT token.
6. Token is stored on the client side.
7. Protected routes verify the token before granting access.
8. Authenticated users can access restricted pages and resources.

## Folder Structure

### Backend Structure

server/

- config/
- controllers/
- middleware/
- models/
- routes/
- server.js

### Frontend Structure

client/

- src/
  - api/
  - context/
  - components/
  - pages/
  - App.jsx
  - main.jsx

## Learning Outcomes

- Learned how JWT authentication works.
- Understood secure password hashing using bcryptjs.
- Implemented authentication middleware in Express.js.
- Learned how to create protected routes in React.
- Improved understanding of client-server communication.
- Gained practical experience with MongoDB and Mongoose.
- Learned state management using Context API.

## Challenges Faced

- Managing authentication state across the application.
- Protecting routes on both frontend and backend.
- Handling JWT token storage and verification.
- Ensuring proper validation and error handling.



These challenges were resolved through middleware implementation, Context API usage, and structured authentication workflows.

## Conclusion

The JWT Authentication Module was successfully implemented using the MERN stack. The application provides secure user registration, login functionality, password encryption, route protection, and session management. This project strengthened my understanding of full-stack authentication systems and secure web application development.